

and preferably a problem for which the quantum representation has a defensible motivation.

Limitations of the experiment

Quantum simulators become expensive as the qubit count increases because a general state-vector simulation grows exponentially. Physical quantum processors introduce noise, restricted qubit connectivity, queueing and finite measurement shots. Classical data must also be encoded into the circuit, and that cost can outweigh any benefit if the representation is not chosen carefully.

Hybrid QML is therefore an active research area rather than a drop-in performance upgrade. PennyLane's strength is that it makes controlled experimentation easier: the same conceptual model can be studied with multiple differentiation techniques, simulators, noise models and supported backends while remaining connected to familiar ML tooling.

Possible extensions are:

- Include Virginica and replace the binary output with a three-class classifier.
- Encode all four Iris features using four qubits or a data re-uploading circuit.
- Compare different embeddings, entanglement patterns and circuit depths.
- Add simulated noise and study whether accuracy and convergence change.
- Compare the hybrid model with logistic regression, SVM and a small classical neural network.
- Execute a suitably compiled version on an accessible hardware backend and compare it with ideal simulation.
- Use repeated train/test splits and report average accuracy with variability, not only one split.

PennyLane occupies a clear position in the open source quantum ecosystem. Qiskit offers a broad

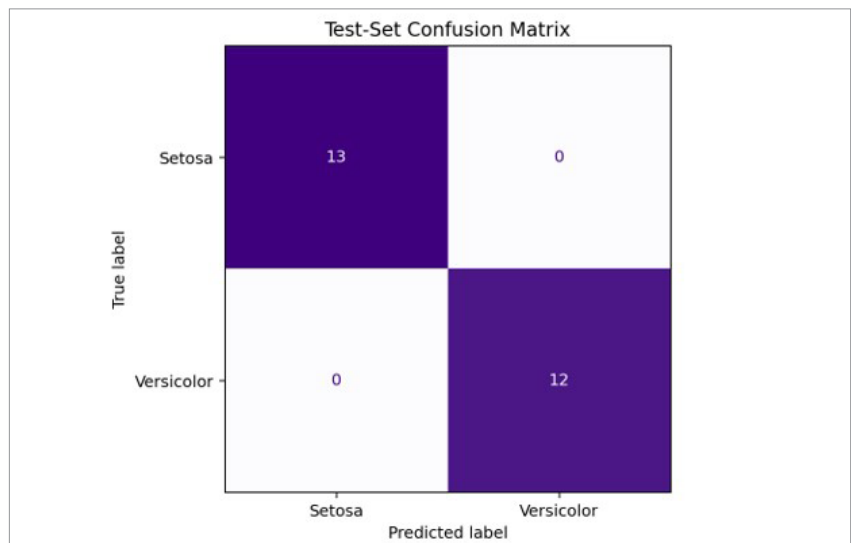


Figure 5(b): Held-out test confusion matrix

IBM-centred development and execution environment; Cirq provides strong circuit and NISQ-device abstractions; TensorFlow Quantum integrates Cirq with TensorFlow. PennyLane distinguishes itself by treating differentiable quantum circuits as trainable building blocks within hybrid applications.

For a newcomer to QML, that design provides a gentle and productive

path: begin on a simulator, encode a small dataset, build a shallow circuit, connect it to PyTorch, observe the training loop and only then explore deeper circuits, noise and hardware. The Iris example does not demonstrate quantum advantage, but shows something equally important for a beginner—the complete route from classical data to qubits and back to a usable prediction. **END** 🐧

References

- PennyLane documentation, <https://docs.pennylane.ai/en/stable/>
- PennyLane PyTorch interface, <https://docs.pennylane.ai/en/stable/introduction/interfaces/torch.html>
- PennyLane gradients and interfaces, <https://docs.pennylane.ai/en/stable/introduction/interfaces.html>
- IBM Quantum and Qiskit documentation, <https://quantum.cloud.ibm.com/docs>
- Google Quantum AI Cirq documentation, <https://quantumai.google/cirq>
- TensorFlow Quantum, <https://www.tensorflow.org/quantum>
- Scikit-learn Iris dataset, https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html

By: S. Ratan Kumar and Valli Kumari Vatsavayi

S. Ratan Kumar is associate professor in the Department of Computer Science and Engineering (AI & ML) at Anil Neerukonda Institute of Technology & Sciences(A) (ANITS), Visakhapatnam. His interests include machine learning, cybersecurity, data analytics, high-performance computing, and the practical teaching of emerging open source technologies.

Valli Kumari Vatsavayi is professor of Computer Science and Systems Engineering, Andhra University. Her research interests include quantum computing, security, privacy, machine learning, and forensics. She has to her credit over 200 research papers.